Rekurencyjne mnożenie macierzy: metody Binét'a i Strassena

Imię Nazwisko (nr albumu) • Imię Nazwisko (nr albumu)

14 października 2026

1 Cel i zakres

Zaimplementowano w C++ dwa rekurencyjne algorytmy mnożenia macierzy kwadratowych o *dowolnym* rozmiarze $n \times n$ (nie tylko będącym potęgą dwójki): metodę Binét'a (8 mnożeń bloków) i metodę Strassena (7 mnożeń bloków). Macierze zawierają liczby losowe z rozkładu równomiernego na przedziale $[10^{-8}, 1)$. Zliczano liczbę operacji zmiennoprzecinkowych $(+, -, *, /)$ oraz mierzono czas działania. Źródła: `binet.cpp`, `strassen.cpp`; skrypty raportowe: `make_plots.py`, `compare.m`.

2 Pseudokod algorytmów rekurencyjnych

W obu metodach macierze dzielimy na cztery bloki $A = \left(\begin{smallmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{smallmatrix}\right)$, analogicznie B i $C = AB$.

Algorytm 1 BINÉT(A, B) (B = wywołanie rekurencyjne)

Wejście: A: $p \times q$, B: $q \times r$
Wyjście: $C = AB$: $p \times r$

jeżeli $p = 1$ lub $q = 1$ lub $r = 1$ **to**
  zwróć MNOŻENIEKLASYCZNE(A, B)
koniec jeżeli
$p_h, q_h, r_h \leftarrow \lceil p/2 \rceil, \lceil q/2 \rceil, \lceil r/2 \rceil$
podziel A w punktach (p_h, q_h), B w (q_h, r_h)
$C_{11} \leftarrow \mathrm{B}(A_{11}, B_{11}) + \mathrm{B}(A_{12}, B_{21})$
$C_{12} \leftarrow \mathrm{B}(A_{11}, B_{12}) + \mathrm{B}(A_{12}, B_{22})$
$C_{21} \leftarrow \mathrm{B}(A_{21}, B_{11}) + \mathrm{B}(A_{22}, B_{21})$
$C_{22} \leftarrow \mathrm{B}(A_{21}, B_{12}) + \mathrm{B}(A_{22}, B_{22})$
zwróć $\left(\begin{smallmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{smallmatrix}\right)$

Algorytm 2 STRASSEN(A, B), $n \times n$ (S = wywołanie rekurencyjne)

jeżeli $n = 1$ **to**
  zwróć $A \cdot B$
koniec jeżeli
jeżeli n nieparzyste, $m = n - 1$ **to**
  wydziel A_{11} $(m \times m)$, kolumnę a_{12}, wiersz a_{21}, liczbę a_{22} (tak samo B)
  $C_{11} \leftarrow \mathrm{S}(A_{11}, B_{11}) + a_{12} b_{21}$
  $C_{12} \leftarrow A_{11} b_{12} + a_{12} b_{22}$
  $C_{21} \leftarrow a_{21} B_{11} + a_{22} b_{21}$
  $C_{22} \leftarrow a_{21} b_{12} + a_{22} b_{22}$
w przeciwnym razie
  $M_1 \leftarrow \mathrm{S}(A_{11}{+}A_{22},\ B_{11}{+}B_{22})$
  $M_2 \leftarrow \mathrm{S}(A_{21}{+}A_{22},\ B_{11})$
  $M_3 \leftarrow \mathrm{S}(A_{11},\ B_{12}{-}B_{22})$
  $M_4 \leftarrow \mathrm{S}(A_{22},\ B_{21}{-}B_{11})$
  $M_5 \leftarrow \mathrm{S}(A_{11}{+}A_{12},\ B_{22})$
  $M_6 \leftarrow \mathrm{S}(A_{21}{-}A_{11},\ B_{11}{+}B_{12})$
  $M_7 \leftarrow \mathrm{S}(A_{12}{-}A_{22},\ B_{21}{+}B_{22})$
  $C_{11} \leftarrow M_1{+}M_4{-}M_5{+}M_7, \quad C_{12} \leftarrow M_3{+}M_5$
  $C_{21} \leftarrow M_2{+}M_4, \quad C_{22} \leftarrow M_1{-}M_2{+}M_3{+}M_6$
koniec jeżeli
zwróć C złożone z $C_{11}, C_{12}, C_{21}, C_{22}$

Dowolny rozmiar. W metodzie Binét'a przy nieparzystych wymiarach bloki górne/lewe mają $\lceil \cdot/2 \rceil$, a dolne/prawe $\lfloor \cdot/2 \rfloor$ wierszy i kolumn, więc mnożymy macierze prostokątne (wzory na C_{ij} pozostają poprawne). W metodzie Strassena dla nieparzystego n „obieramy" ostatni wiersz i kolumnę (*dynamic peeling*); rdzeń $(n{-}1) \times (n{-}1)$ mnożymy rekurencyjnie, a cienkie iloczyny klasycznie. Macierzy nie dopełniamy zerami, dzięki czemu liczba operacji nie jest sztucznie zawyżona. Rekurencję prowadzimy do macierzy 1×1 (bez progu przełączenia na metodę klasyczną).

3 Najważniejsze fragmenty kodu

Licznik operacji zwiększany jest tylko w dwóch miejscach: przy dodawaniu/odejmowaniu macierzy (1 operacja na element) oraz w mnożeniu klasycznym używanym w przypadku bazowym (pqr mnożeń i $pr(q{-}1)$ dodawań).

```cpp
static unsigned long long g_flops = 0;           // licznik operacji + - * /
Matrix add(const Matrix& A, const Matrix& B) {   // sub() analogicznie
    Matrix C(A.rows, A.cols);
    for (size_t k = 0; k < C.data.size(); ++k) C.data[k] = A.data[k] + B.data[k];
    g_flops += C.data.size();                    // 1 operacja na element
    return C;
}
```

Binét (`binet.cpp`, fragment skrócony):

```cpp
Matrix multiplyBinet(const Matrix& A, const Matrix& B) {
    const int p = A.rows, q = A.cols, r = B.cols;
    if (p == 1 || q == 1 || r == 1) return multiplyNaive(A, B);   // przypadek bazowy
    const int ph = (p + 1) / 2, qh = (q + 1) / 2, rh = (r + 1) / 2;  // punkty podzialu
    Matrix A11 = subMatrix(A, 0, 0, ph, qh), A12 = subMatrix(A, 0, qh, ph, q - qh);
    Matrix A21 = subMatrix(A, ph, 0, p - ph, qh), A22 = subMatrix(A, ph, qh, p - ph, q - qh);
    // ... B11, B12, B21, B22 wycinane analogicznie (wiersze wg qh, kolumny wg rh)
    Matrix C11 = add(multiplyBinet(A11, B11), multiplyBinet(A12, B21));   // 8 mnozen
    Matrix C12 = add(multiplyBinet(A11, B12), multiplyBinet(A12, B22));
    Matrix C21 = add(multiplyBinet(A21, B11), multiplyBinet(A22, B21));
    Matrix C22 = add(multiplyBinet(A21, B12), multiplyBinet(A22, B22));
    // ... sklejenie C11..C22 w macierz wynikowa p x r (placeBlock)
}
```

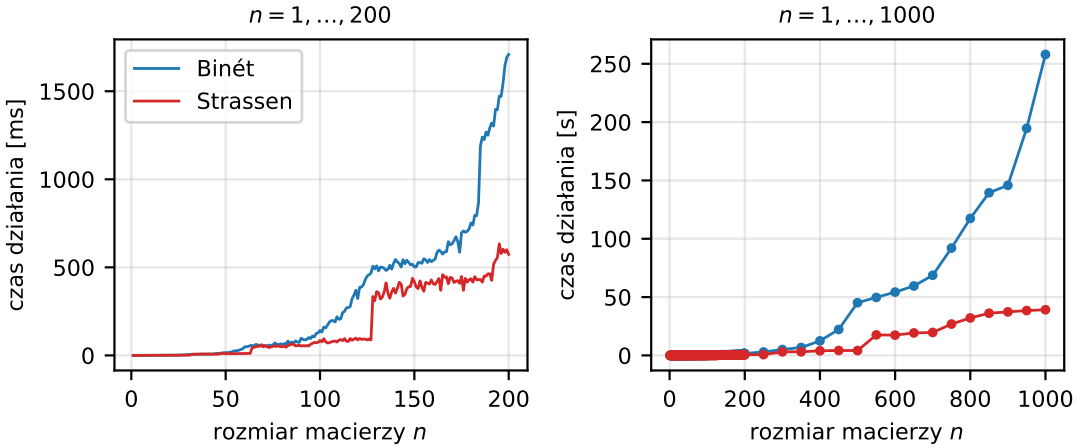
Strassen (`strassen.cpp`, fragment skrócony):

```cpp
Matrix multiplyStrassen(const Matrix& A, const Matrix& B) {
    const int n = A.rows;
    if (n <= STRASSEN_CUTOFF) return multiplyNaive(A, B);   // CUTOFF = 1
    if (n % 2 == 1) {                                        // obieranie: m = n-1 parzyste
        // ... A11 (m x m), a12, a21, a22 oraz b.. wycinane przez subMatrix
        Matrix C11 = add(multiplyStrassen(A11, B11), multiplyNaive(a12, b21));
        // ... C12, C21, C22: tylko "cienkie" iloczyny multiplyNaive + add
    }
    // n parzyste: 7 mnozen rekurencyjnych zamiast 8
    Matrix M1 = multiplyStrassen(add(A11, A22), add(B11, B22));
    Matrix M2 = multiplyStrassen(add(A21, A22), B11);
    Matrix M3 = multiplyStrassen(A11, sub(B12, B22));
    Matrix M4 = multiplyStrassen(A22, sub(B21, B11));
    Matrix M5 = multiplyStrassen(add(A11, A12), B22);
    Matrix M6 = multiplyStrassen(sub(A21, A11), add(B11, B12));
    Matrix M7 = multiplyStrassen(sub(A12, A22), add(B21, B22));
    Matrix C11 = add(sub(add(M1, M4), M5), M7);   Matrix C12 = add(M3, M5);
    Matrix C21 = add(M2, M4);                     Matrix C22 = add(add(sub(M1, M2), M3), M6);
    // ... sklejenie C11..C22 w macierz wynikowa n x n
}
```

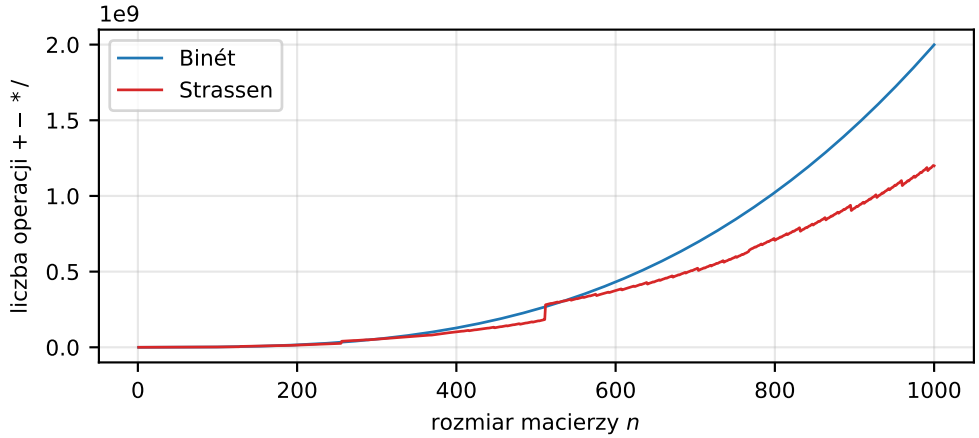
4 Środowisko pomiarowe

Język nie jest interpretowany: C++ jest kompilowany z góry do natywnego kodu maszynowego x86-64 (`g++ 14.2.0`, MinGW-W64, flagi `-O2 -std=c++17`); powstały plik `.exe` jest wykonywany bezpośrednio przez procesor, bez interpretera i maszyny wirtualnej. Obliczenia były jednowątkowe. **Sprzęt:** procesor AMD Ryzen 7 7735HS (8 rdzeni / 16 wątków, taktowanie bazowe 3,2 GHz), pamięć RAM 31,2 GB DDR5 (4800 MT/s), system Windows 11. Czas mierzono zegarem `std::chrono::steady_clock` tylko dla samego mnożenia (bez generowania macierzy i weryfikacji): dla $n \leq 200$ jako minimum z 3 powtórzeń, dla $n > 200$ jako pojedynczy pomiar (programy uruchamiane kolejno; dla $n \leq 200$ oba programy liczyły równocześnie na osobnych rdzeniach). Pomiary czasu wykonano dla $n = 1, 2, \ldots, 200$ oraz $n = 250, 300, \ldots, 1000$ (krok 50); liczbę operacji wyznaczono dla *każdego* $n = 1, \ldots, 1000$, korzystając z modelu rekurencyjnego (to samo drzewo wywołań co w algorytmie, ale bez mnożenia macierzy). Dla wszystkich zmierzonych n model daje liczbę operacji identyczną z licznikiem w działającym programie. Maksymalny rozmiar, który udało się policzyć obiema metodami: $n = 1000$.

5 Wykresy



Rysunek 1: Czas działania obu metod rekurencyjnych w zależności od rozmiaru macierzy n.



Rysunek 2: Liczba operacji zmiennoprzecinkowych $(+, -, *, /)$ w zależności od n, $n = 1, \ldots, 1000$.

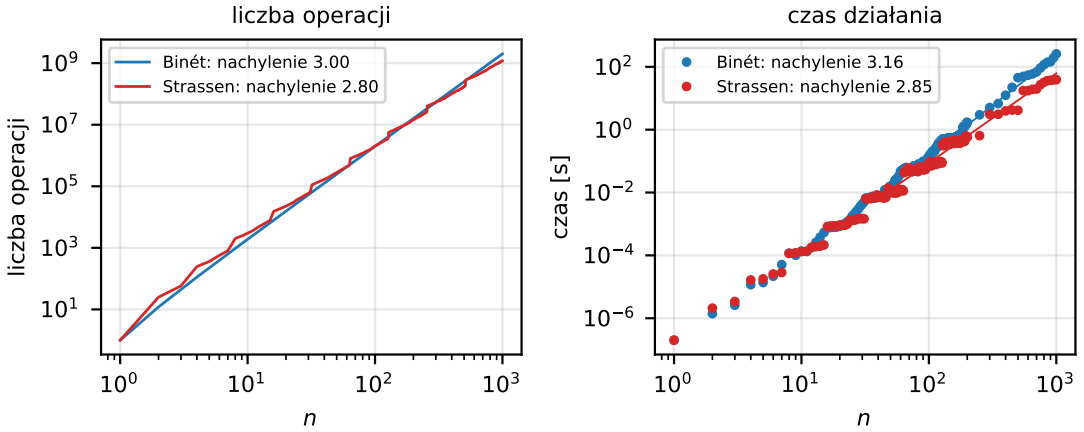
Komentarz. Liczba operacji w metodzie Binét'a wynosi dokładnie $2n^3 - n^2$ (dla $n = 1000$: 1 999 000 000), czyli tyle samo co w algorytmie klasycznym. Strassen dla małych n wykonuje *więcej* operacji (1,92× dla $n = 16$, 1,01× dla $n = 100$), bo rekurencja aż do macierzy 1×1 generuje wiele dodawań macierzy; dla $n \geq 536$ ma już zawsze mniej operacji niż Binét (dla $n = 1000$: 1 199 169 602, czyli 60% operacji Binét'a). Wykres Strassena jest „schodkowy": tuż po przekroczeniu potęgi dwójki (np. $n = 511 \to 512$: z $1{,}83 \cdot 10^8$ do $2{,}81 \cdot 10^8$ operacji) rekurencja schodzi o jeden poziom głębiej. Czas działania zależy nie tylko od liczby operacji, ale też od kopiowania i alokacji macierzy tymczasowych oraz od pamięci podręcznej – stąd skoki na rys. 1. Dla $n = 1000$ Strassen potrzebuje ok. 39 s, a Binét ok. 258 s (6,6 razy dłużej), mimo że liczby operacji różnią się tylko 1,7 raza: Binét tworzy więcej macierzy tymczasowych (8 wywołań rekurencyjnych na poziom zamiast 7).

6 Oszacowanie złożoności obliczeniowej

Teoretycznie. Binét: $T(n) = 8T(n/2) + \Theta(n^2)$, więc z twierdzenia o rekurencji uniwersalnej $T(n) = \Theta(n^{\log_2 8}) = \Theta(n^3)$; dokładnie $2n^3 - n^2$ operacji (zgodnie z danymi dla każdego $n = 1, \ldots, 1000$). Strassen: $S(n) = 7S(n/2) + 18(n/2)^2$, więc $S(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2{,}807})$; dla $n = 2^k$ dokładnie $S(n) = 7\,n^{\log_2 7} - 6\,n^2$ (sprawdzone dla $k = 0, \ldots, 9$; np. $n = 4$: 247 operacji).

Eksperymentalnie (rys. 3): dopasowanie prostej do danych w skali log-log daje wykładnik α w zależności $\sim n^\alpha$. Dla liczby operacji: Binét $\alpha = 3{,}00$, Strassen $\alpha = 2{,}80$ ($n \geq 16$, wszystkie n; dla samych $n = 2^k$ wychodzi 2,87, nieco więcej niż 2,807, bo dla małych n ważny jest składnik $-6n^2$). Dla czasu ($n \geq 50$): Binét $\alpha = 3{,}16$, Strassen $\alpha = 2{,}85$. Czas rośnie nieco szybciej niż liczba operacji ze względu na efekty pamięci. **Wniosek:** Binét

ma złożoność $O(n^3)$, Strassen $O(n^{\log_2 7}) \approx O(n^{2{,}81})$; przewaga Strassena rośnie z n, ale przy rekurencji do 1×1 ujawnia się dopiero dla n rzędu kilkuset.



Rysunek 3: Wykresy w skali log-log; nachylenie dopasowanej prostej oznacza wykładnik α w zależności $\sim n^{\alpha}$.

7 Przykład 4×4 – porównanie z Octave/MATLAB

Do porównania wybrano $A =$ `magic(4)`, $B = A^T/7$ (te same macierze są zaszyte w trybie `--example` obu programów). Skrypt `compare.m` wczytuje wyniki programów C++ i porównuje je z $A * B$ w Octave/MATLAB. Wynik referencyjny:

$$C_{\text{ref}} = AB \approx \begin{pmatrix} 62{,}5714 & 33{,}7143 & 47{,}4286 & 21{,}4286 \\ 33{,}7143 & 44{,}2857 & 39{,}7143 & 47{,}4286 \\ 47{,}4286 & 39{,}7143 & 44{,}2857 & 33{,}7143 \\ 21{,}4286 & 47{,}4286 & 33{,}7143 & 62{,}5714 \end{pmatrix}.$$

metoda	liczba operacji	$\max \lvert C - C_{\text{ref}} \rvert$
Binét	112 $(= 2 \cdot 4^3 - 4^2)$	$7{,}1 \cdot 10^{-15}$
Strassen	247 $(= 7 \cdot 7^2 - 6 \cdot 4^2)$	$1{,}4 \cdot 10^{-14}$

Obie metody dają wynik zgodny z mnożeniem wbudowanym z dokładnością do błędów zaokrągleń (rzędu 10^{-14}); Strassen ma nieco większy błąd, bo wykonuje więcej dodawań/odejmowań w innej kolejności.